

Midterm Project (UTS) — Project Description & Requirements

Web and Mobile Application Development *Team assignment · covers one continuous topic from Midterm through Final*

1. The Big Picture

By the **Final Project**, each team will deliver one complete system built around your assigned topic. That system has three parts:

- **A Web application** — used by two kinds of people: the **Administrator** (the service provider — e.g. hospital staff, restaurant owner, event organizer, depending on your topic) and the **Client** (the person using the service).
- **A Mobile application** — used only by the **Client**.
- **One shared backend and database** — both the Web app and the Mobile app read and write the same data.

For the Midterm (UTS), you will only build the Web application, using hardcoded (fake) data. There is no backend, no database, and no Mobile app yet. This is not a simplified or "fake" version of the project — it is genuinely the first stage of the same system you will finish at Final. The reason it's scoped this way is purely about what you've been taught so far: backend development, databases, real authentication, and mobile development are all covered later in the course. Once you've learned them, you'll come back and connect this same Web app to a real backend, and then build the Mobile app on top of it.

2. Your Team & Topic

Your team and topic stay the same from Midterm all the way to Final — this is one project done in two stages, not two separate projects. Refer to the topic sheet you were given (e.g. Hospital Appointment, Food Ordering, Event Management, etc.) for your assigned domain.

3. User Roles — Read This Carefully

Every topic has exactly two roles:

Role	Who they are	Where they exist
Administrator	The service provider side (e.g. doctor/staff, restaurant/admin, organizer)	Web only
Client	The service user side (e.g. patient, customer, participant)	Web and Mobile

At Final, the Web app supports login for *both* roles, while the Mobile app is *Client-only*. For the Midterm, since there is no Mobile app yet, you need to represent **both roles inside your Web app**.

4. Midterm (UTS) Deliverable — What to Build

Build **3–4 related web pages**:

1. **Login page**
2. **Menu / navigation**
3. **Dashboard**
4. **One form page** (e.g. an "add" or "edit" form relevant to your topic — a new appointment, a new order, a new registration, etc.)

Data source: hardcoded in your frontend code (a JavaScript array or object). No database, no backend API calls of any kind.

Important — you do NOT need to build two separate sets of pages for the two roles. Keep it to the 3–4 pages above. Instead, have your Menu and Dashboard **conditionally render different content depending on which role is currently logged in** — the same idea you've already used for showing/hiding UI based on state. An Administrator's dashboard might show management-style information; a Client's dashboard shows their own information. Same page, same component, different content based on role.

5. About the "Login" — Read This Before You Build It

The login for this stage is a **simulation only**, not real authentication:

- Keep a hardcoded list of users in your code, each with a username, password, and a **role** field ("administrator" or "client").
- On submit, check the entered username/password against this list. If it matches, store the matched user's role in state and navigate to the Menu/Dashboard.
- That's it. There is no password hashing, no token, no server-side check, no session.

Do not implement real authentication — no JWT, no third-party auth libraries (Firebase Auth, Auth0, etc.), no encryption. That content is taught later in the course, and using it now means two things go wrong: you're working outside what you've learned, and you likely won't be able to explain your own code on request — which fails the explain-your-code requirement.

6. What Is *Not* Required at Midterm

To be explicit, none of the following are expected right now:

- A backend / API (Express) — comes later
- A real database (SQL) — comes later
- Real authentication or security — comes later
- A Mobile app — comes later
- Anything beyond static hosting — deploy your Web app to **GitHub Pages**, same as before

If your team has already built any of these ahead of schedule on your own initiative, that's fine — but it is not required.

7. Your Own Short Project Description (submit this with your code)

Along with your deployed link and your repository, each team submits a short written description — a page or less. Include:

- **Project name** and a one-paragraph description of what your system does.
- **What the Administrator can do** in your system.
- **What the Client can do** in your system.
- **The pages you built** for the Midterm and what each one shows.
- **A rough data sketch** — for the 1–2 main things your topic revolves around (e.g. an "appointment" or an "order"), list the fields you imagine that record would have. This does not need to be technical or final, it's a first draft of your data model, and it's fine if it changes later.

8. How This Is Graded

- The core flow works (login → menu/dashboard → form, for both roles).
- All relevant states are handled — loading, empty, and error states where they apply.
- Your commit history is genuine — real incremental commits, not one large upload.
- You can explain any part of your own submitted code on request.
- **Not graded on:** visual polish, or how many extra features you add.

9. Looking Ahead — Plan With This in Mind

What you build now is the foundation of your Final system, not a throwaway exercise:

- In the coming weeks, you'll learn to build your own backend (Express) and database (SQLite), and you'll reconnect this same Web app to real data instead of the hardcoded list.
- Later, you'll learn mobile development and build the Client-only Mobile app on top of that same backend.
- Your Final deliverable = Web app (Administrator + Client) + Mobile app (Client only) + shared backend + one AI-powered feature, fully deployed.

A little foresight now — thinking about what fields your data will eventually need, and keeping your components reasonably organized — will save your team rework later. But don't over-build for the future at the expense of finishing what's due now.

10. Summary Timeline

Stage	What's due	Data source
Midterm (UTS)	Web app: login, menu, dashboard, 1 form (both roles represented via conditional rendering)	Hardcoded

Stage	What's due	Data source
<i>(mid-course, not separately graded)</i>	Web app reconnected to your own real backend	Real database
Final	Web app (Admin + Client) + Mobile app (Client only) + shared backend + AI feature, fully deployed	Real database